Path Algebras of Quivers as a Sparse Multi-Hop Attention for Mitigating Over-Squashing in GNNs

Carlos Isaac Zainea Maya[1][0000-0002-6459-2397] and Agustín Moreno Cañadas[1][0000-0001-6812-5131]

Universidad Nacional de Colombia, Departamento de Matemáticas, Bogotá, Colombia
{cizaineam,amorenoca}@unal.edu.co

Abstract. Message-passing graph neural networks (GNNs) suffer from *over-squashing*: when information must travel along many long paths into a node, it is compressed into a fixed-width vector and the signal is lost. We introduce **Walk Attention**, an architecture that addresses over-squashing through the *path algebra* $\Bbbk Q$ of the directed graph (quiver) underlying the data, whose graded components give an exact, algebraic description of multi-hop reachability. We use this structure to define a sparse, multi-hop attention: the path algebra supplies the attention *support* (which vertices a node reaches by a length-k walk) and the network learns the *weights*. On bottleneck graphs with tunable over-squashing we prove the separation—message passing with linear aggregation is at exactly chance accuracy for every depth and width, while a single Walk Attention layer at the matching range solves the retrieval task exactly—and confirm it empirically: Walk Attention reaches 1.000 ± 0.000 accuracy over five seeds where one-hop graph attention degrades toward chance and a fixed-weight aggregator only partially succeeds; it stays exact on the standard RingTransfer probe and, under a shared small training budget, is comparable to one-hop attention on the real-world Peptides-func benchmark—its advantage is decisive when over-squashing is severe and modest when it is mild. A negative result motivates the design: using the path-algebra *quotient* to collapse equivalent walks discards multiplicity the task needs and fails, so the algebra is valuable not for compression but as the object that *defines the attention support*. Conceptually, Walk Attention is the learned, representational reading of an algebraic substrate whose dynamical reading is the automata-of-impact framework.

Keywords: Graph neural networks · Over-squashing · Path algebra · Quiver · Representation theory · Attention.

1 Introduction

Graph neural networks (GNNs) built on the message-passing paradigm [10] update each node by aggregating messages from its immediate neighbours and iterating this local rule across layers. After k layers, information from nodes k hops

away reaches a target node. While simple and effective, this scheme has a well-documented failure mode: *over-squashing* [3]. When the number of paths carrying information into a node grows—typically exponentially with distance—the messages along all of those paths must be compressed into a single fixed-width vector. The receiving node cannot recover them individually, and long-range signal is lost. Over-squashing is the principal obstacle to learning tasks whose labels depend on distant or globally-distributed information.

Existing remedies act on the graph or the aggregation: *rewiring* adds or reweights edges to widen bottlenecks [17], graph *coarsening* and *pooling* reduce the graph while trying to preserve structure, and curvature-based analyses explain *where* squashing occurs [6]. A complementary line of work replaces local aggregation by global *attention* as in Transformers [18], at the cost of an all-pairs interaction that ignores graph structure.

In this paper we take a different route, grounded in the representation theory of quivers. A *quiver* is a directed graph, possibly with multiple arrows [15,4], and its *path algebra* $\Bbbk Q$ has the directed walks of the quiver as a basis, with concatenation as multiplication. The graded piece $e_j \cdot \Bbbk Q_k \cdot e_i$ of this algebra has a basis consisting of the directed walks of length k from vertex i to vertex j; its dimension equals $(A^k)_{ij}$, the corresponding entry of the k-th power of the adjacency matrix. The path algebra therefore provides an *exact, algebraic description of multi-hop reachability and multiplicity.*

Our central observation is that this algebraic structure is precisely what is needed to define a principled, sparse, multi-hop attention. We let each node attend directly to every vertex reachable from it by a walk of length k: the path algebra supplies the *support* of the attention (which pairs may interact at range k), and the network learns the *weights* (how much). Because the walk-reachability support is sparse—typically a small fraction of all node pairs—this is far cheaper than full self-attention, while still acting at a distance and thereby bypassing the iterative bottleneck that causes over-squashing. We call the resulting architecture **Walk Attention**.

This places Walk Attention on a common algebraic ground (Section 3.7): since a graph network on Q is a representation of the quiver ($\mathrm{Rep}(Q) \cong \Bbbk Q\text{-Mod}$), it is the *learned* reading of a substrate whose *dynamical* reading is the automata-of-impact-over-quivers framework [23].

Contributions.

1. We give a *self-contained* construction (Section 3) of quivers, path algebras, graded components, the impact-degree neighbourhood system, and the associated *walk operators*, developing the needed representation theory from first principles for a pattern-recognition audience.
2. We define **Walk Attention** (Section 4): a learned multi-hop attention whose support is the path-algebra walk-reachability structure, and we relate it to message passing and to Transformer attention.
3. We *prove* the separation on the bottleneck family (Section 4.4): with the standard one-layer-per-hop depth, any message-passing network whose first

aggregation is linear in the source features (GCN, GIN) has exactly chance accuracy for every depth, width, and parameter setting, whereas a single Walk Attention layer at the matching range solves the retrieval task exactly.
4. On a controlled bottleneck family with tunable over-squashing (Section 5), Walk Attention attains 1.000 ± 0.000 accuracy over five seeds where one-hop graph attention degrades toward chance and a fixed-weight multi-hop aggregator only partially succeeds; on the real Peptides-func benchmark, under a shared small training budget, it is comparable to one-hop attention, so its advantage is regime-dependent (decisive under severe over-squashing, modest when it is mild).
5. We report a *negative* result that sharpens the design: collapsing equivalent walks via the path-algebra *quotient* discards needed multiplicity and fails. The quotient's role is to *determine* the support, not to compress the signal.

2 Related Work

Over-squashing. Alon and Yahav [3] identified over-squashing and proposed the NeighborsMatch probe, on which standard GNNs degrade sharply with the problem radius. Topping et al. [17] connected over-squashing to negative graph curvature and proposed stochastic discrete Ricci flow (SDRF) rewiring; Di Giovanni et al. [6] gave a sensitivity analysis relating squashing to width, depth, and topology. These works either modify the graph or characterise the phenomenon. We instead change the aggregation operator, attending at a distance over an algebraically-defined support.

Attention and graph Transformers. Graph attention networks [19] learn one-hop attention weights; full graph Transformers apply Transformer self-attention [18] over all node pairs, recovering long range at quadratic cost and with positional/structural encodings to reinject topology. Walk Attention is intermediate between the two: it is attention, but its support is the sparse multi-hop walk-reachability of the quiver rather than one hop or all pairs.

Multi-hop aggregation and attention. Aggregating beyond one hop is well explored: MixHop [2] and SIGN [8] mix fixed powers of (normalised) adjacency operators, graph diffusion convolution [9] rewires by a diffusion kernel, MAGNA [20] diffuses attention scores over multiple hops, shortest-path message passing aggregates over k-hop distance shells [1], NAGphormer [5] tokenises hop-wise aggregates for a Transformer, Graphormer [21] biases dense attention by shortest-path distance, and DRew [11] rewires message passing dynamically with delay. Walk Attention differs in the object that defines the support: the graded piece of the path algebra—directed *walks* with their multiplicity, not distance shells or diffusion weights—which makes the support exact (Proposition 1), exposes the quotient $\Bbbk Q/I$ as a design degree of freedom (Section 5), and embeds the architecture in the representation theory of quivers (Section 3.7).

Path and walk structure. Path-counting and powers of the adjacency matrix are classical, and the path algebra of a quiver is a standard object in representation theory [15,4]. Brauer-configuration algebras and related path-algebraic constructions have recently been applied to combinatorial and applied problems by Moreno Cañadas and collaborators [14,13]. Closest to the present work is the impact-degree neighbourhood system and the automata-of-impact-over-quivers framework [23,12], which we take as the dynamical sibling of Walk Attention over the same algebraic substrate (Section 3.7). Attention over multi-hop supports is thus not new in itself; what is new, to our knowledge, is taking the support from the *graded path algebra* of the quiver, with its multiplicities, its quotient, and its dynamical (automata) reading.

3 The Path Algebra of a Quiver

This section is self-contained. We build, from first principles, the algebraic objects that the model in Section 4 relies on: quivers, paths, the path algebra, its grading by length, the impact-degree neighbourhood system, and the *walk operators* that encode multi-hop reachability. We develop the needed representation theory from first principles, assuming no prior exposure; standard references are [15,4].

3.1 Quivers and paths

Definition 1 (Quiver). *A* quiver *is a quadruple $Q = (Q_0, Q_1, s, t)$ where Q_0 is a finite set of* vertices*, Q_1 is a finite set of* arrows*, and $s, t\colon Q_1 \to Q_0$ assign to each arrow α its* source *$s(\alpha)$ and* target *$t(\alpha)$. We write $\alpha\colon i \to j$ when $s(\alpha) = i$ and $t(\alpha) = j$.*

A quiver is thus a directed graph in which multiple arrows between the same ordered pair of vertices are allowed (a *multigraph*). For pattern recognition this is exactly the data of a directed relational structure: vertices are entities and arrows are directed relations, with parallel arrows recording multiplicity. We write $n = |Q_0|$ and identify Q_0 with $\{1, \dots, n\}$.

Definition 2 (Path). *A* path of length $k \geq 1$ *in Q is a sequence of arrows $p = \alpha_k \cdots \alpha_2 \alpha_1$ such that $t(\alpha_r) = s(\alpha_{r+1})$ for $1 \leq r < k$; that is, the arrows compose head-to-tail. Its source is $s(p) = s(\alpha_1)$ and its target is $t(p) = t(\alpha_k)$. For each vertex i we also admit a* trivial path *e_i of length 0 with $s(e_i) = t(e_i) = i$.*

We read paths right-to-left, as for function composition. A path is a directed *walk*: vertices and arrows may repeat. The number of distinct walks of a given length between two vertices is the quantity that drives over-squashing, and the path algebra makes it algebraically explicit.

3.2 The path algebra and its grading

Let $\Bbbk$ be a field (in practice $\Bbbk = \mathbb{R}$).

Definition 3 (Path algebra). *The* path algebra $\Bbbk Q$ *is the* $\Bbbk$*-vector space with basis the set of all paths of* Q *(including the trivial paths* e_i*), endowed with the bilinear multiplication given on basis paths by concatenation:*

$$q \cdot p \;=\; \begin{cases} q\,p & \text{if } s(q) = t(p), \\ 0 & \text{otherwise,} \end{cases}$$

and extended linearly. The trivial paths satisfy $e_{t(p)}\,p = p = p\,e_{s(p)}$ *and* $e_i e_j = \delta_{ij} e_i$*, so* $\{e_i\}_{i \in Q_0}$ *is a complete set of orthogonal idempotents and* $1 = \sum_i e_i$ *is the unit.*

The multiplication respects path length: concatenating a path of length k with one of length ℓ yields a path of length $k+\ell$ (or zero). Hence $\Bbbk Q$ is a *graded* algebra,

$$\Bbbk Q \;=\; \bigoplus_{k \geq 0} \Bbbk Q_k, \qquad \Bbbk Q_k \cdot \Bbbk Q_\ell \subseteq \Bbbk Q_{k+\ell},$$

where $\Bbbk Q_k$ is the subspace spanned by the paths of length exactly k. In particular $\Bbbk Q_0 = \bigoplus_i \Bbbk e_i$ and $\Bbbk Q_1$ is spanned by the arrows.

The idempotents e_i let us isolate the paths between a fixed pair of vertices. For $i, j \in Q_0$ and $k \geq 0$, the subspace

$$e_j \cdot \Bbbk Q_k \cdot e_i \;\subseteq\; \Bbbk Q$$

is spanned exactly by the paths of length k from i to j. Its dimension is therefore the *number of directed walks* of length k from i to j. The following elementary proposition states the correspondence with linear algebra precisely; it underlies the model's computation.

Proposition 1 (Graded multiplicity equals walk count). *Let* $A \in \mathbb{N}^{n \times n}$ *be the adjacency matrix of* Q*,* $A_{ij} = \#\{\alpha \in Q_1 : \alpha \colon i \to j\}$*. Then for all* $k \geq 0$ *and all* $i, j \in Q_0$*,*

$$\dim_{\Bbbk}\big(e_j \cdot \Bbbk Q_k \cdot e_i\big) \;=\; (A^{\,k})_{ij}.$$

Proof. By induction on k. For $k = 0$, $e_j \Bbbk Q_0 e_i = \delta_{ij}\,\Bbbk e_i$, of dimension $\delta_{ij} = (A^0)_{ij} = (\mathrm{Id})_{ij}$. For $k = 1$, a basis of $e_j \Bbbk Q_1 e_i$ is the set of arrows $i \to j$, of cardinality A_{ij}. Assume the claim for $k-1$. Every path of length k from i to j factors uniquely as $\alpha \cdot p$, where p is a path of length $k-1$ from i to some intermediate vertex r and $\alpha \colon r \to j$ is an arrow. The number of such factorisations is $\sum_r A_{rj}\,(A^{k-1})_{ir} = (A^{k})_{ij}$, which proves the claim. □

We call the integer $n_k(i,j) := (A^{\,k})_{ij} = \dim_{\Bbbk}(e_j \Bbbk Q_k e_i)$ the *walk multiplicity* from i to j at range k. It is exactly the number of length-k messages that a message-passing GNN forces through node j from node i after k layers, and hence the source of over-squashing.

3.3 Impact degree and the fundamental neighbourhood system

The grading by length induces a canonical stratification of the vertices around each node, which organises the multi-hop structure the model will exploit. For a quiver Q, the *impact degree* $g(i,j)$ is the length of the shortest directed path from i to j (with $g(i,i)=0$ and $g(i,j)=\infty$ if no directed path exists); equivalently, $g(i,j)=\min\{k : (A^k)_{ij} > 0\}$, the first range at which the walk multiplicity becomes positive.

Definition 4 (Fundamental neighbourhood system). *For a node $c \in Q_0$, its* in-neighbourhood *of radius k is*

$$A_k(c) \;=\; \{\, i \in Q_0 : g(i,c) \le k \,\},$$

the set of nodes that can send influence to c along a directed path of length at most k. The nested family $\{A_k(c)\}_{k\ge 0}$, with $A_0(c)=\{c\} \subseteq A_1(c) \subseteq \cdots$, is the fundamental neighbourhood system *(FNS) of c, and the layer $\mathcal{N}_k(c) = A_k(c) \setminus A_{k-1}(c)$ collects the nodes at impact degree exactly k.*

The FNS is the combinatorial object that the walk operators of Definition 5 act on. The precise relation between its layers and walk-reachability is the following.

Proposition 2 (FNS layers vs. walk reachability). *For $k \ge 1$ let $\mathcal{S}_k = \{(c,i) : (A^k)_{ic} > 0\}$ be the pairs joined by a length-k walk $i \rightsquigarrow c$. Then*

$$\{(c,i) : i \in \mathcal{N}_k(c)\} \;\subseteq\; \mathcal{S}_k,$$

and the inclusion is strict in general: a length-k walk may also reach nodes at impact degree $< k$. If Q is graded *—there is $\rho\colon Q_0 \to \mathbb{Z}$ with $\rho(t(\alpha)) = \rho(s(\alpha))+1$ for every arrow α—then equality holds for every k.*

Proof. If $i \in \mathcal{N}_k(c)$ then $g(i,c)=k$ and a shortest path is a length-k walk, so $(A^k)_{ic} > 0$. Strictness: in the quiver $1 \to 2 \to 3$ with an extra arrow $1 \to 3$ one has $(A^2)_{13} > 0$ yet $g(1,3)=1$. If Q is graded, every walk $i \rightsquigarrow c$ has length $\rho(c)-\rho(i)$; a length-k walk forces $\rho(c)-\rho(i)=k$, so every walk—in particular a shortest path—has length k, giving $g(i,c)=k$ and $i \in \mathcal{N}_k(c)$. □

The benchmark graphs of Section 5 are graded (the layer index is such a ρ), so there the supports coincide with the FNS layers; on graphs with cycles, such as the ring, $\mathcal{S}_k$ strictly contains the layer relation. Stacking ranges $k = 1, 2, \ldots$ therefore walks the embedding outward through the layers of the FNS, revisiting earlier layers where the topology permits. The multiplicity $n_k(i,c)$ records *how many* length-k paths populate each layer—the textured, algebraic refinement of the scalar impact degree.

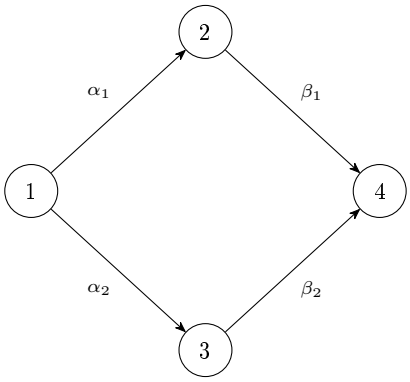

Fig. 1. The diamond quiver of Example 1. The two length-2 walks $1 \rightsquigarrow 4$ are parallel; $\dim(e_4 \Bbbk Q_2 e_1) = 2$.

3.4 An explicit example

Example 1 (A diamond quiver). Let Q have vertices $\{1, 2, 3, 4\}$ and arrows $\alpha_1 \colon 1 \to 2$, $\alpha_2 \colon 1 \to 3$, $\beta_1 \colon 2 \to 4$, $\beta_2 \colon 3 \to 4$ (Figure 1). Its adjacency matrix and the square are

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad A^2 = \begin{pmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

There are two length-2 walks from 1 to 4, namely $\beta_1\alpha_1$ and $\beta_2\alpha_2$, so $\dim(e_4 \Bbbk Q_2 e_1) = (A^2)_{14} = 2$, in agreement with Proposition 1. These two walks are *parallel*: same source, same target, same length. Whether they should be treated as one channel or two is a modelling decision we return to in Section 5.

3.5 Walk operators

Proposition 1 lets us turn the graded path algebra into a family of linear operators on node features, one per range k. These *walk operators* are the bridge to the neural model.

Definition 5 (Walk operator). *For each range $k \geq 1$ define the matrix $W^{(k)} \in \mathbb{R}^{n \times n}$ by*

$$W^{(k)}_{ji} \;=\; \frac{(A^k)_{ij}}{Z_k},$$

where $Z_k = \max_j \sum_i (A^k)_{ij}$ is a single normalising constant for the range. The walk-reachability support *at range k is the index set*

$$\mathcal{S}_k \;=\; \big\{(j,i) : (A^k)_{ij} > 0\big\} \;=\; \big\{(j,i) : \dim(e_j \Bbbk Q_k e_i) > 0\big\}.$$

Acting on a feature matrix $H \in \mathbb{R}^{n\times d}$, the operator $W^{(k)}H$ sends to each node j a weighted sum of the features of every node i that reaches j by a length-k walk, the weight being proportional to the walk multiplicity $n_k(i,j)$. Two facts are worth stressing.

(i) Global normalisation, not row normalisation. We divide by a single constant Z_k per range rather than normalising each row to sum to one. Row normalisation would map the multiplicities to a uniform distribution over the reachable sources and would erase the very multiplicity information that distinguishes $W^{(k)}$ from a plain reachability mask; the global constant keeps the relative multiplicities intact while bounding the magnitude.

(ii) Sparsity. The support $\mathcal{S}_k$ is exactly the nonzero pattern of A^k. On the structured graphs of interest it is a small fraction of all n^2 pairs (see Section 5), so $W^{(k)}$ can be stored and applied in $O(|\mathcal{S}_k|)$ rather than $O(n^2)$.

The fixed-weight operator $W^{(k)}$ already mitigates over-squashing, because it lets node j read from range-k sources *directly*, in one step, instead of forcing the signal through k rounds of local message passing. Its limitation—which motivates Walk Attention—is that the weights are fixed by multiplicity and cannot *select* which reachable source matters. We address this in Section 4 by learning the weights over the same support.

3.6 The quotient algebra

Representation theory routinely passes from $\Bbbk Q$ to a quotient $\Bbbk Q/I$ by an *admissible ideal* I, identifying paths that are declared equal. In the diamond of Example 1, the relation $\beta_1\alpha_1 \equiv \beta_2\alpha_2$ generates an ideal I for which $\dim\big(e_4(\Bbbk Q/I)_2 e_1\big) = 1$: the two parallel walks collapse to a single class. More generally, $\dim\big(e_j(\Bbbk Q/I)_k e_i\big) \le n_k(i,j)$, so the quotient yields a structurally smaller multiplicity.

It is tempting to use this quotient directly as an over-squashing remedy: replace the walk multiplicities $n_k(i,j)$ by the smaller quotient dimensions, thereby "de-duplicating" redundant paths before aggregation. We tested this and found it *counterproductive* (Section 5): collapsing parallel walks destroys multiplicity information that downstream tasks may require. The constructive role of the quotient, then, is not to compress the signal but to *reshape the support*—e.g. to route distinct path classes to distinct attention heads—a direction we leave to future work. In the model that follows we use the path algebra only through the reachability support $\mathcal{S}_k$ and the (unquotiented) walk operators $W^{(k)}$.

3.7 A common algebraic ground

The objects above—the graded path algebra $\Bbbk Q$, the impact degree and its fundamental neighbourhood system (FNS), and the quotient $\Bbbk Q/I$—form one algebraic substrate that specialises in two directions, according to how a node-update map φ over the layers $\mathcal{N}_k(c)$ is read. In the *dynamical* reading, φ is a fixed evolution rule applied over the FNS, weighting influence by impact degree; iterating it in time yields the *automata of impact over quivers* (AIQ) framework,

with its SI/SIS/SIR rules and category structure, developed separately [23,12]. In the *representational* reading, taken here, φ is *learned*: a vector space at each vertex and a linear map at each arrow is a representation of Q (a $\Bbbk Q$-module, via $\mathrm{Rep}(Q) \cong \Bbbk Q\text{-Mod}$ [15,4]), and a graph neural network on Q is such a representation with learned arrow maps. Walk Attention is its graded instance: a learned representation whose support is the FNS. The two readings share the trunk and differ only in whether φ is prescribed or learned.

4 Walk Attention

We now define the architecture. The guiding principle is the observation that the multi-hop walk operator of Section 3 is structurally an *attention*: it aggregates, for each target node, a weighted combination of the features of the nodes that reach it. Transformer self-attention [18] has the same shape but with all-pairs support and *learned* weights; graph attention [19] restricts the support to one hop. Walk Attention keeps the learned weights of attention and takes the support from the path algebra: the multi-hop, sparse walk-reachability $\mathcal{S}_k$. In the terms of Section 3.7, it is the learned node update φ of the common substrate—a graded representation of Q whose arrow maps are optimised over the fundamental neighbourhood system.

4.1 The Walk Attention layer

Fix a range k with reachability support $\mathcal{S}_k$ (Definition 5). A Walk Attention layer with H heads and head dimension C transforms node features $x \in \mathbb{R}^{n \times d_{\mathrm{in}}}$ as follows. For each head h it computes queries, keys and values by linear maps,

$$Q^h = x\,W_Q^h, \qquad K^h = x\,W_K^h, \qquad V^h = x\,W_V^h, \qquad W_\bullet^h \in \mathbb{R}^{d_{\mathrm{in}} \times C},$$

and then attends *only over the reachable pairs*: for every $(j,i) \in \mathcal{S}_k$,

$$\ell_{ji}^h = \frac{1}{\sqrt{C}}\,\langle Q_j^h,\, K_i^h \rangle, \tag{1}$$

$$\alpha_{ji}^h = \mathrm{softmax}_{\,i:\,(j,i)\in\mathcal{S}_k}\big(\ell_{ji}^h\big), \tag{2}$$

$$z_j^h = \sum_{i:\,(j,i)\in\mathcal{S}_k} \alpha_{ji}^h\, V_i^h. \tag{3}$$

The softmax in (2) is taken, for each target j, over exactly the sources i that reach j by a length-k walk—a *segmented* softmax over the support, never over all n nodes. The head outputs are concatenated (or averaged in the final layer) and a residual "root" term is added:

$$\mathrm{WA}_k(x)_j \;=\; \big\|_{h=1}^{H} z_j^h \;+\; x_j W_{\mathrm{root}}.$$

Because (1)–(3) only ever touch the pairs in $\mathcal{S}_k$, the layer costs $O(|\mathcal{S}_k|\,H\,C)$ time and memory, not $O(n^2)$. The support is precomputed once per graph from Proposition 1 (the nonzero pattern of A^k). Figure 2 summarises the data flow.

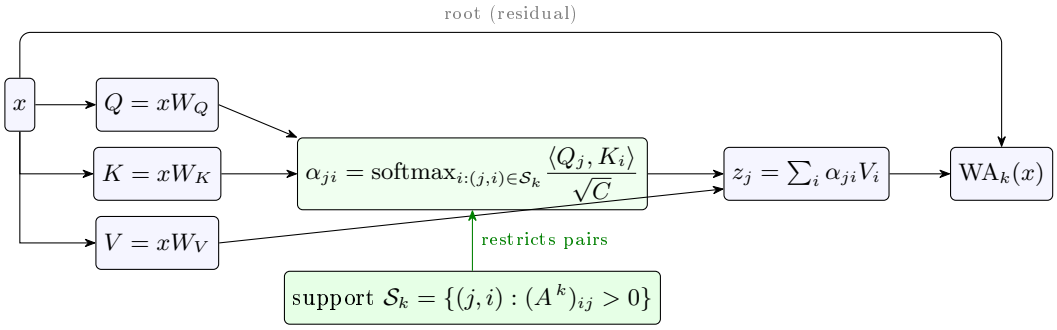

Fig. 2. One Walk Attention layer at range k. Queries, keys and values are linear maps of the node features; attention is computed *only* over the walk-reachable pairs $\mathcal{S}_k$ supplied by the path algebra (the nonzero pattern of A^k), then aggregated and added to a residual root term. The green support box is what distinguishes Walk Attention from dense ($\mathcal{S}_k$ = all pairs) and one-hop ($\mathcal{S}_k$ = edges) attention.

Remark 1 (Why learned weights matter). The fixed-weight operator $W^{(k)}$ of Definition 5 is the special case in which α^h_{ji} is replaced by the constant $n_k(i,j)/Z_k$. It reaches every range-k source but weights them by multiplicity alone, so it cannot *select* which source carries the relevant signal. The learned attention (1)–(2) restores this selectivity while keeping the same multi-hop support. Section 5 shows this distinction is decisive.

4.2 The network

A depth-L Walk Attention network stacks one layer per range $k = 1, \ldots, L$. Layer k uses the support $\mathcal{S}_k$ derived from A^k, so successive layers read from progressively longer walks. Each layer is followed by layer normalisation, an ELU nonlinearity and dropout; a final multilayer perceptron maps the node embedding to task outputs:

$$x^{(k)} \;=\; \mathrm{dropout}\Big(\mathrm{elu}\big(\mathrm{LN}\big(\mathrm{WA}_k\big(x^{(k-1)}\big)\big)\big)\Big), \qquad \hat{y} \;=\; \mathrm{MLP}\big(x^{(L)}\big).$$

Layer normalisation is important in practice: without it the learned attention is high-variance across initialisations (it reaches the optimum on some seeds and not others); with it, together with a short learning-rate warmup and gradient clipping, training converges reliably (Section 5). Algorithm 1 summarises one layer.

4.3 Relation to message passing and Transformers

Table 1 situates Walk Attention among related operators by two axes: the *support* of interaction and whether the weights are *learned*. A one-hop graph-attention layer [19] learns weights but only over direct neighbours, so it must propagate

Algorithm 1 Walk Attention layer at range k (sparse).

Require: features $x \in \mathbb{R}^{n \times d_{\text{in}}}$; support $\mathcal{S}_k = \{(j,i) : (A^k)_{ij} > 0\}$
$Q, K, V \leftarrow xW_Q,\ xW_K,\ xW_V$ ▷ per head; shapes $n \times HC$
for each $(j,i) \in \mathcal{S}_k$ and head h **do**
  $\ell^h_{ji} \leftarrow \langle Q^h_j, K^h_i \rangle / \sqrt{C}$
end for
$\alpha^h_{ji} \leftarrow$ segmented-softmax over $\{i : (j,i) \in \mathcal{S}_k\}$
$z^h_j \leftarrow \sum_i \alpha^h_{ji} V^h_i$ ▷ scatter-add into targets
return $\Big\|_h z^h \ + \ xW_{\text{root}}$

Table 1. Walk Attention against related aggregation operators.

Operator	Support	Weights	Cost
GCN / message passing	1-hop	fixed (degree)	$O(\lvert Q_1\rvert)$
Graph attention (GAT)	1-hop	learned	$O(\lvert Q_1\rvert)$
Walk operator $W^{(k)}$	walk-reachable	fixed (multiplicity)	$O(\lvert\mathcal{S}_k\rvert)$
Graph Transformer	all pairs	learned	$O(n^2)$
Walk Attention (ours)	**walk-reachable**	**learned**	$O(\lvert\mathcal{S}_k\rvert)$

long-range signal through L stacked layers—precisely the regime where over-squashing is most severe. A full graph Transformer [18] learns weights over all pairs, paying $O(n^2)$ and discarding graph structure unless it is reinjected by positional encodings. The fixed-weight walk operator has the right multi-hop support but cannot select. Walk Attention occupies the remaining cell: multi-hop, structure-aware support *and* learned, selective weights.

4.4 A formal account on the bottleneck family

On the bottleneck-chain family used in Section 5 the separation between one-hop message passing and Walk Attention can be *proved*, not only measured. Recall the construction: K source vertices carry a one-hot *key*—the assignment of keys to sources is a uniformly random permutation π—and a one-hot *value*; d bottleneck layers $L_1, \ldots, L_d$ of M interchangeable, featureless vertices follow, consecutive layers fully connected; a target vertex carries a one-hot *query* q and must output the value of the source whose key equals q (label $\pi^{-1}(q)$, chance $1/K$). Throughout, networks have the standard depth $L = d + 1$—one layer per hop, as in the NeighborsMatch protocol [3] and in our experiments—and hidden width w.

Proposition 3 (Layer collapse). *For any message-passing network on this graph whose update* $h'_v = U\big(h_v, \mathrm{AGG}\{h_u : u \to v\}\big)$ *uses the same* U *and* AGG *at every vertex: after every round, all* M *vertices of each bottleneck layer carry identical states. Consequently the entire influence of the sources on the target passes through a single* w*-dimensional state per layer—while the number of length-*$(d{+}1)$ *walks carrying it is* KM^d *(Proposition 1).*

Proof. Induction on the round. Vertices of a bottleneck layer start with identical (zero) features, and two vertices of the same layer have the same in-multiset of neighbours (all sources for L_1; all of L_{r-1} for L_r), whose states are equal by induction; shared U, AGG produce equal updates. □

Proposition 4 (Linear first aggregation is permutation-blind). *Assume in addition that messages are linear in the sender's state with coefficients independent of feature content—as in GCN (degree-normalised linear messages) and GIN (sum of neighbour states). Then with $L = d + 1$ layers the target's output depends on the sources only through $\sum_s x_s$, which is the* same for every π*: the key slots of a permutation sum to the all-ones vector. The expected accuracy is therefore exactly $1/K$, for every depth d, width w, and choice of parameters.*

Proof. In-neighbours of the target are L_d (and itself); unrolling, the target's state after round $d{+}1$ depends on the sources only through the state of L_1 after round 1: source information entering L_1 at round t needs d further hops to reach the target, arriving at round $t + d \le d + 1$ only if $t \le 1$. At round 1 the message from source s is a fixed linear map of the raw feature x_s, with one shared coefficient (all sources are isomorphic in the graph), so the state of L_1 is a function of $\sum_s x_s$. With keys $e_{\pi(s)}$, the key block of $\sum_s x_s$ equals $\sum_s e_{\pi(s)} = (1, \ldots, 1)$ for every π, and the value block is likewise π-independent; hence the output is independent of π. Given the query q, the label $\pi^{-1}(q)$ is uniform on $\{1, \ldots, K\}$, so any π-independent prediction has expected accuracy $1/K$. □

Proposition 5 (One Walk Attention layer suffices). *At range $k = d{+}1$ the support of the target is exactly the set of K sources (the graph is graded; Proposition 2). Choose W_Q to project the query slot scaled by any $\beta > 0$, W_K the key slot, W_V the value slot, and $W_{root} = 0$. Then the Walk Attention output at the target is $z_t = \sum_s \alpha_s \, e_{v(s)}$ whose largest entry sits at the coordinate of the correct value, on* every *instance and for all d and M.*

Proof. The logit of source s is $\beta \langle e_q, e_{\pi(s)} \rangle = \beta \, \mathbf{1}[\pi(s) = q]$, so the softmax over the K reachable sources puts weight $e^\beta / (e^\beta + K - 1)$ on the unique matching source s^* and equal smaller weight on the rest. The values $e_{v(s)}$ are distinct basis vectors, so the largest coordinate of z_t is α_{s^*}, at the coordinate $v(s^*) = \pi^{-1}(q)$, the label. □

Remark 2. The propositions delimit the mechanisms seen in Section 5. GAT escapes Proposition 4—its first-round coefficients depend on source content—but must still thread the key–value association through the single-vector channel of Proposition 3, re-encoded $d{+}1$ times: empirically it sits above chance and degrades with depth. The fixed-weight walk operator reads the sources directly but with *equal* weights (all multiplicities equal M^d), so it must embed the whole key–value multiset in w dimensions rather than select one source: it plateaus in between. Walk Attention both reads directly and selects (Proposition 5), and solves the task.

5 Experiments

We evaluate Walk Attention on a controlled family of graphs in which over-squashing is severe and *tunable*, so that the contribution of each architectural component can be isolated. All code, configurations and the experiment logs are released with the paper.

5.1 A tunable over-squashing benchmark

Bottleneck-chain graphs. We build a family parameterised by a *depth* d, a number of sources K and a width M. The graph has K source vertices, then d consecutive *bottleneck layers* of M interchangeable vertices each, and finally one target vertex. Every source connects to every vertex of the first layer, every vertex of layer r connects to every vertex of layer $r+1$, and every vertex of the last layer connects to the target. Because the bottleneck vertices are interchangeable, the number of length-$(d{+}1)$ walks from a source to the target is M^d, so the walk multiplicity into the target grows as $n_{d+1}(\cdot, \text{target}) \sim K\,M^d$ (Proposition 1); a message-passing GNN must compress all of them into the target's fixed-width vector. The walk-reachability support, by contrast, remains sparse: at the deepest range it consists of only the K source–target pairs ($5/196 \approx 2.6\%$ of all node pairs at depth 2 and $5/324 \approx 1.5\%$ at depth 3 in our configuration).

Task. Each source carries a one-hot *key* and a scalar *payload*; the target carries a query key. The model must output, at the target node, the key of the source whose key matches the query—a retrieval task whose answer lies $d{+}1$ hops away, beyond the bottleneck. Accuracy is measured at the target node; chance level is $1/K$.

Models. We compare:

- **GAT** [19]: one-hop graph attention, $d{+}1$ layers (learned weights, one-hop support);
- **Walk operator** (`walkraw`): the fixed-weight multi-hop operator $W^{(k)}$ of Definition 5 (multi-hop support, fixed multiplicity weights);
- **Walk Attention** (ours): Algorithm 1 (multi-hop support, learned weights).

All models use hidden width 16, four heads where applicable, and $d{+}1$ layers/ranges. We train with AdamW, a ten-epoch linear learning-rate warmup followed by cosine decay, and gradient clipping; Walk Attention additionally uses layer normalisation after each layer. We report mean and standard deviation of target accuracy on a held-out validation split (the synthetic task has no separate test set) over five random seeds, at problem depths $d \in \{2, 3\}$ with $K = 5$, $M = 4$.

Table 2. Target accuracy (mean ± std over 5 seeds) on bottleneck-chain retrieval, K=5, M=4. Chance is $1/K = 0.20$. Depth d is the number of bottleneck layers; the answer lies d+1 hops away. GCN and GIN sit at chance, as Proposition 4 mandates (the residual +0.01–0.02 over $1/K$ is best-epoch selection on a finite validation split).

Model	**depth** $d = 2$	**depth** $d = 3$
GCN (one-hop, linear aggregation)	0.21 ± 0.01	0.22 ± 0.01
GIN (one-hop, linear aggregation)	0.21 ± 0.01	0.21 ± 0.01
GAT (one-hop attention)	0.45 ± 0.22	0.29 ± 0.17
Walk operator (fixed weights)	0.62 ± 0.12	0.50 ± 0.12
Walk Attention (ours)	$\mathbf{1.000 \pm 0.000}$	$\mathbf{1.000 \pm 0.000}$

5.2 Main result

Table 2 reports the results, which land exactly where Propositions 3–5 put them (Remark 2). One-hop GAT degrades sharply with depth, approaching chance: a local operator cannot propagate the signal across the bottleneck under over-squashing. GCN and GIN, whose first aggregation is linear, are pinned at chance by Proposition 4—and sit there empirically at every depth. The fixed-weight walk operator reaches the target—its multi-hop support spans the bottleneck in a single layer—but, weighting all reachable sources by multiplicity alone, it cannot select the matching source and plateaus at 0.50–0.62. **Walk Attention solves the task exactly**, attaining 1.000 ± 0.000 accuracy at both depths over all five seeds: the same multi-hop support, now with learned query–key weights, lets the target attend selectively to the relevant source.

On stability. Without layer normalisation and warmup, Walk Attention has a high ceiling but is high-variance: across seeds its accuracy ranged from 0.44 to 1.00. The three standard stabilisers (layer normalisation, learning-rate warmup, gradient clipping) remove this variance entirely, yielding the ± 0.000 in Table 2. The instability was therefore an optimisation artefact, not a property of the operator.

5.3 RingTransfer

The bottleneck family is our own construction. To confirm the effect on an *independent* benchmark we use RingTransfer [6]: a cycle of n nodes in which a source must transmit its one-hot label to the diametrically opposite target. The parallel-path structure is explicit: the source reaches the target along the *two arcs* of the ring, two paths of equal length $n/2$. Over-squashing is severe and tunable: every node has two neighbours, so the number of length-$(n/2)$ walks entering the target—the messages a one-hop network must compress after $n/2$ layers—grows as $2^{n/2}$ with the ring size, while only the two arc walks carry the signal. The task requires $n/2$ hops, so a one-hop model requires $n/2$ layers. We use five classes (chance 0.2), hidden width 32, the same optimiser recipe as above

Table 3. RingTransfer accuracy vs. ring size n (distance $n/2$), mean $\pm$ std over 5 seeds; chance = 0.20. One-hop GCN/GAT collapse as the ring—and thus the over-squashing—grows, while Walk Attention stays exact.

Model	n=6	n=10	n=14	n=18
GCN	1.00 ± 0.00	1.00 ± 0.00	0.81 ± 0.13	0.53 ± 0.33
GAT	1.00 ± 0.00	1.00 ± 0.00	0.69 ± 0.29	0.29 ± 0.19
Walk Attention (ours)	$\mathbf{1.00 \pm 0.00}$	$\mathbf{1.00 \pm 0.00}$	$\mathbf{1.00 \pm 0.00}$	$\mathbf{1.00 \pm 0.00}$

Table 4. De-duplicating walks via the quotient $\Bbbk Q/I$ *hurts*: target accuracy (mean $\pm$ std over 5 seeds), bottleneck retrieval, under the same protocol as Table 2 (the walk-operator row coincides with it). The quotient operator is clearly below the unquotiented one at both depths.

Operator	**depth** $d = 2$	**depth** $d = 3$
Quotient $\Bbbk Q/I$ (collapsed walks)	0.39 ± 0.04	0.30 ± 0.09
Walk operator (raw walks)	$\mathbf{0.62 \pm 0.12}$	$\mathbf{0.50 \pm 0.12}$

(AdamW, linear warmup, cosine decay, gradient clipping), and $n/2$ layers/ranges for all models.

Table 3 reports the results. On small rings ($n \leq 10$, distance ≤ 5) every model succeeds, as the squashing is not yet severe. From $n = 14$ (distance 7) onward the one-hop baselines degrade and become highly seed-dependent—at $n = 18$ GAT falls to 0.29 ± 0.19 and GCN to 0.53 ± 0.33—while **Walk Attention remains at** 1.000 ± 0.000 **throughout**: its multi-hop support aggregates the source directly along either arc, and the learned weights identify the labelled source. The gap widens with ring size, i.e. with the severity of over-squashing, as predicted.

5.4 Collapsing walks hurts

The path-algebra quotient $\Bbbk Q/I$ of Section 3 suggests an alternative remedy: replace the walk multiplicities $n_k(i, j)$ by the smaller quotient dimensions $\dim(e_j(\Bbbk Q/I)_k e_i)$, de-duplicating equivalent walks before aggregation. We implemented this (the fixed-weight operator built from the quotient dimensions) and compared it to the unquotiented walk operator on the same task, in a controlled ablation that changes *only* the operator (quotient vs. raw) and holds everything else—data, architecture, and the optimiser recipe of Table 2—fixed. As Table 4 shows, **the quotient operator falls clearly below the raw one at both depths**. Collapsing parallel walks discards the source multiplicity that the retrieval task needs: in this regime redundant paths are *signal*, not noise, and removing them removes information.

This negative result indicates that the contribution of the path algebra here is *not* compression. Its appropriate role is to supply the multi-hop attention *support*—sparsely and exactly—over which the network learns weights, which is precisely Walk Attention. Whether a regime exists in which the quotient's

compression is itself beneficial (e.g. when parallel paths are genuinely redundant noise, or when distinct path classes are routed to distinct heads) remains an open question.

5.5 LRGB Peptides-func

The synthetic benchmark is deliberately a worst case: an extreme bottleneck where multi-hop reach is decisive. To assess Walk Attention on real data we also evaluate on **Peptides-func** from the Long Range Graph Benchmark [7] (10,873/2,331/2,331 train/val/test molecular graphs, median 138 nodes; multi-label graph classification, metric Average Precision). We use the same atom embedding, four layers, hidden width 80, mean pooling and AdamW for every model; Walk Attention uses ranges $k = 1, \ldots, 4$ with the walk-reachability support of each molecule. On these graphs the support stays sparse (1–8% of n^2 at range 3), so the method remains cheap.

Table 5 reports test AP. **Walk Attention is comparable to the one-hop baselines but does not surpass them here**: it exceeds GCN and is statistically indistinguishable from GAT (0.526 ± 0.007 vs. 0.526 ± 0.004 over four seeds). Two caveats frame this comparison. First, all models share a deliberately small budget (four layers, width 80, no positional encodings, a short schedule), so our absolute numbers are below published results for this benchmark—the LRGB paper reports GCN at 0.593 under its tuned 500k-parameter protocol [7], and subsequent re-evaluations are higher still [16]; the comparison is therefore *internal*, isolating the aggregation operator under an identical protocol, not a claim against the state of the art. Second, the result is not explained by an absence of parallel paths: peptide graphs contain rings and branches, and a typical molecule has hundreds of vertex pairs joined by ≥ 2 walks of length ≤ 4 (which the quotient ideal would collapse). Rather, the over-squashing is *mild*: these graphs are small (median 138 nodes), so a few layers of one-hop attention already reach far enough, and the multiplicity into any node does not grow large enough to saturate the embedding. The multi-hop support that is decisive under severe over-squashing yields little benefit when the squashing is mild. Walk Attention's advantage is therefore *regime-dependent*—large when the squashing is severe (Tables 2 and 3), and comparable to one-hop attention when it is mild—while on real data the operator continues to train stably at moderate cost (about $1.6\times$ the wall-clock of one-hop GAT, averaged over seeds, far below dense all-pairs attention).

5.6 Cost

Walk Attention operates on the walk-reachability support, whose size is the number of nonzero entries of A^k. On the bottleneck family this is a small fraction of n^2 (1.5–2.6% at the deepest range, depending on the depth), so the layer is far cheaper than a dense graph Transformer while retaining its long range. The supports are precomputed once per graph topology and cached, so the path-algebra computation does not enter the training loop.

Table 5. LRGB Peptides-func, test Average Precision (mean ± std over 4 seeds; higher is better), under a shared small budget (four layers, width 80, no positional encodings) that isolates the aggregation operator; published results under the standard tuned 500k-parameter protocol are higher for every architecture [7,16]. Walk Attention ties GAT and exceeds GCN; on this mild-over-squashing benchmark the multi-hop support yields no decisive gain (contrast Table 2).

Model	**test AP**
GCN	0.492 ± 0.011
GAT	0.526 ± 0.004
Walk Attention (ours)	0.526 ± 0.007

6 Conclusion and Future Work

We introduced **Walk Attention**, an architecture that mitigates over-squashing in graph neural networks by attending over a multi-hop support supplied by the path algebra of the underlying quiver. The graded path algebra gives, through Proposition 1, an exact account of which node pairs are connected by length-k walks and with what multiplicity; we use this to define a sparse attention whose support is algebraically determined and whose weights are learned. On the bottleneck family the separation is proved (Section 4.4): message passing with linear first aggregation is at exactly chance accuracy for every depth and width, while a single Walk Attention layer at the matching range solves the task exactly. On a tunable bottleneck benchmark, Walk Attention solved a long-range retrieval task exactly (1.000 ± 0.000 over five seeds) where a one-hop attention network degraded toward chance and a fixed-weight multi-hop aggregator only partially succeeded; on the real Peptides-func benchmark, under a shared small training budget, it was comparable to one-hop attention while remaining sparse and stable. Its advantage is regime-dependent—decisive when over-squashing is severe and comparable to one-hop attention when it is mild. A complementary negative result showed that using the path-algebra *quotient* to compress equivalent walks is counterproductive when path multiplicity carries signal; the role of the algebra is to determine the attention support, not to discard information.

The construction is deliberately elementary and self-contained, so that it can serve as an entry point connecting quiver representation theory to graph representation learning. Walk Attention is the learned, representational reading of a common algebraic substrate (Section 3.7)—the graded path algebra and its impact-degree neighbourhood system—whose dynamical reading is the automata-of-impact-over-quivers framework [23]; relating the two is itself a direction. Several others follow naturally.

- **From learned weights to dynamics.** The dynamical sibling [23] fixes the node-update rule φ *a priori*, weighting influence by impact degree; Walk Attention instead *learns* those weights. This suggests a two-way bridge: the attention weights learned on a task can be injected into an AIQ as a data-driven

evolution rule, letting one read off the *dynamical* effect of each weight—how strongly a given source, at a given impact degree, drives the state of a node over time—and, conversely, letting AIQ stability and reachability analyses interpret and constrain what Walk Attention has learned. Quantifying this correspondence between learned attention and impact-weighted dynamics is, to us, the most promising direction.

- **Benchmarks with severe over-squashing.** Peptides-func is a mild case where the multi-hop support is not decisive. Identifying real long-range tasks whose graphs do contain acute bottlenecks—where the synthetic advantage is expected to carry over—and comparing against rewiring [17], multi-hop [11] and graph-Transformer baselines, is the natural next step.
- **Quotient-shaped supports.** The quotient $\Bbbk Q/I$, rather than compressing the signal, can *partition* walks into equivalence classes; routing distinct classes to distinct attention heads would let the algebra structure the heads themselves. Identifying which relations I are useful—and learning them from data, much as the dynamical sibling fixes them by a rule—is an appealing problem.
- **Scaling the support.** For dense graphs the range-k support can itself grow large; truncating, sampling, or learning a sparser admissible support are natural ways to control cost while preserving long range.

Reproducibility. All graph generators, the path-algebra walk operators, the Walk Attention layer, the baselines, and the experiment-tracking logs are released [24]. The path-algebra computations build on the open-source `aiq-quivers` library [22]; the present work is, to our knowledge, the first published application of that algebraic machinery to graph neural networks, and we have therefore kept the algebraic construction explicit throughout.

References

1. Abboud, R., Dimitrov, R., Ceylan, İ.İ.: Shortest path networks for graph property prediction. In: Learning on Graphs Conference (LoG) (2022)
2. Abu-El-Haija, S., Perozzi, B., Kapoor, A., Alipourfard, N., Lerman, K., Harutyunyan, H., Ver Steeg, G., Galstyan, A.: MixHop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In: International Conference on Machine Learning (ICML) (2019)
3. Alon, U., Yahav, E.: On the bottleneck of graph neural networks and its practical implications. In: International Conference on Learning Representations (ICLR) (2021)
4. Assem, I., Simson, D., Skowroński, A.: Elements of the Representation Theory of Associative Algebras: Volume 1. Cambridge University Press (2006)
5. Chen, J., Gao, K., Li, G., He, K.: NAGphormer: A tokenized graph transformer for node classification in large graphs. In: International Conference on Learning Representations (ICLR) (2023)
6. Di Giovanni, F., Giusti, L., Barbero, F., Luise, G., Lio, P., Bronstein, M.M.: On over-squashing in message passing neural networks: The impact of width, depth,

and topology. In: Proceedings of the 40th International Conference on Machine Learning (ICML). PMLR, vol. 202, pp. 7865–7885 (2023)
7. Dwivedi, V.P., Rampášek, L., Galkin, M., Parviz, A., Wolf, G., Luu, A.T., Beaini, D.: Long range graph benchmark. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track. vol. 35 (2022)
8. Frasca, F., Rossi, E., Eynard, D., Chamberlain, B., Bronstein, M., Monti, F.: SIGN: Scalable inception graph neural networks. arXiv preprint arXiv:2004.11198 (2020)
9. Gasteiger, J., Weißenberger, S., Günnemann, S.: Diffusion improves graph learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2019)
10. Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E.: Neural message passing for quantum chemistry. In: Proceedings of the 34th International Conference on Machine Learning (ICML). PMLR, vol. 70, pp. 1263–1272 (2017)
11. Gutteridge, B., Dong, X., Bronstein, M.M., Di Giovanni, F.: DRew: Dynamically rewired message passing with delay. In: International Conference on Machine Learning (ICML) (2023)
12. Ibáñez Huertas, J.A., Zainea Maya, C.I.: CAsimulations: Modelado de la dinámica topológica en una enfermedad mediante autómatas celulares. Comunicaciones en Estadística **18**(1) (2025). https://doi.org/10.15332
13. Moreno Cañadas, A., Rodríguez-Nieto, J.G., Salazar Díaz, O.P.: Brauer configuration algebras induced by integer partitions and their applications in the theory of branched coverings. Mathematics **12**(22), 3626 (2024). https://doi.org/10.3390/math12223626
14. Osorio Angarita, M.A., Moreno Cañadas, A.: Brauer configuration algebras for multimedia based cryptography and security applications. Multimedia Tools and Applications **80**(15), 23485–23510 (2021). https://doi.org/10.1007/s11042-020-10239-3
15. Schiffler, R.: Quiver Representations. Springer (2014)
16. Tönshoff, J., Ritzert, M., Rosenbluth, E., Grohe, M.: Where did the gap go? Reassessing the long-range graph benchmark. Transactions on Machine Learning Research (2024), arXiv:2309.00367
17. Topping, J., Di Giovanni, F., Chamberlain, B.P., Dong, X., Bronstein, M.M.: Understanding over-squashing and bottlenecks on graphs via curvature. In: International Conference on Learning Representations (ICLR) (2022)
18. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 30 (2017)
19. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. arXiv preprint arXiv:1710.10903 (2017)
20. Wang, G., Ying, R., Huang, J., Leskovec, J.: Multi-hop attention graph neural networks. In: International Joint Conference on Artificial Intelligence (IJCAI) (2021)
21. Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., Liu, T.Y.: Do transformers really perform badly for graph representation? In: Advances in Neural Information Processing Systems (NeurIPS) (2021)
22. Zainea Maya, C.I., Moreno Cañadas, A.: aiq-quivers: a Python library for quivers, path algebras, and impact automata. https://github.com/Izainea/aiq-quivers (2026), version 1.2.1
23. Zainea Maya, C.I., Moreno Cañadas, A.: Automata of impact over quivers (2026), manuscript in preparation
24. Zainea Maya, C.I., Moreno Cañadas, A.: path-algebra-gnn: Walk attention — code and experiments. https://github.com/Izainea/path-algebra-gnn (2026)